

Programación Vectorial en Ensamblador x86-64 (NASM/Linux)

Normalizador estadístico vectorizado: comparación escalar vs. AVX2

Ana Cristina Fallas Quirós

07/10/2026

Repositorio / entrega: https://github.com/AnaFallas/Arqui_Proyecto_1.git
Curso: Arquitectura de Computadores

Índice

1. Introducción y objetivos	2
2. Entorno de pruebas	2
3. Implementación escalar	3
3.1. <code>sum_array</code> (escalar)	3
3.2. <code>compute_stats</code> (escalar)	3
3.3. <code>normalize_array</code> (escalar)	4
4. Implementación vectorial	5
4.1. Justificación de AVX2 frente a otras extensiones	5
4.2. <code>sum_array</code> (vectorial)	5
4.3. Reducción horizontal: estrategia elegida	6
4.4. <code>compute_stats</code> (vectorial)	6
4.5. Manejo del remanente	7
4.6. <code>normalize_array</code> (vectorial)	7
4.7. Caso de estudio: bug de corrupción de acumulador	7
5. Casos de prueba	9
6. Resultados de rendimiento	9
6.1. Tiempos de ejecución	9
6.2. Gráfico de speedup vs. N	10
6.3. <code>perf stat</code>	10
6.4. Interpretación	11
7. Evidencia de la sesión de GDB	12
8. Conclusiones, limitaciones y trabajo futuro	13
A. Diagrama de arquitectura	14

1. Introducción y objetivos

El presente proyecto implementa un *normalizador estadístico* que, dado un arreglo grande de números en punto flotante de precisión simple, calcula sus estadísticos descriptivos (suma, media, varianza, desviación estándar, mínimo y máximo) y produce una versión normalizada (z-score) del arreglo: $y[i] = (x[i] - \mu)/\sigma$. El interés del ejercicio no está en el cálculo estadístico en sí — trivial en cualquier lenguaje de alto nivel — sino en implementarlo dos veces en ensamblador x86-64: una vez con instrucciones escalares (SISD) y otra con instrucciones vectoriales AVX2 (SIMD), procesando 8 `float` por iteración, para medir y explicar cuantitativamente la diferencia de rendimiento entre ambos modelos de ejecución.

Los objetivos de aprendizaje que este informe demuestra son los siguientes:

- **OA1–OA2.** Explicar el modelo SIMD frente al escalar clásico y ubicar AVX2 en la evolución histórica de las extensiones vectoriales de x86 (MMX, SSE, AVX, AVX-512).
- **OA3.** Escribir funciones de ensamblador x86-64 en NASM que usen registros `XMM/YMM` e instrucciones empaquetadas, respetando la convención de llamada System V AMD64 ABI.
- **OA4.** Aplicar alineación de memoria y resolver el remanente ($n \bmod 8 \neq 0$) con un bucle escalar de cierre.
- **OA5.** Usar GDB para inspeccionar registros `YMM` y memoria, y depurar errores típicos (en particular, la corrupción de un acumulador documentada en la Sección 4.7).
- **OA6.** Medir y comparar rendimiento (tiempo, speedup, IPC, cache-misses) entre escalar y vectorial, e interpretar los resultados a la luz del ancho de banda de memoria y la Ley de Amdahl.
- **OA7.** Comunicar y defender oralmente las decisiones de diseño e implementación de bajo nivel.

2. Entorno de pruebas

La máquina de pruebas es una laptop Dell Inspiron 3501 con CPU Intel Core i5-1135G7 @ 2.40 GHz (11.^a generación, Tiger Lake). Se confirmó soporte AVX2 mediante `lscpu | grep avx2`, requisito indispensable para ejecutar la versión vectorial.

Ítem	Valor
CPU (modelo)	Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
Soporte AVX2 confirmado	Sí (flag <code>avx2</code> en <code>lscpu</code>)
NASM	2.16.01
GCC	13.3.0 (Ubuntu 13.3.0-6ubuntu2 24.04.1)
GDB	15.1 (Ubuntu 15.1-1ubuntu1 24.04.1)
Sistema operativo	Ubuntu 24.04.4 LTS (noble)

Cuadro 1: Entorno de pruebas utilizado.

3. Implementación escalar

La versión escalar implementa los tres kernels usando exclusivamente instrucciones SSE escalares (movss, addss, subss, mulss, divss, comiss), procesando un float por iteración.

3.1. sum_array (escalar)

Recorre el arreglo elemento por elemento, acumulando en xmm0.

```
1 sum_array:
2     xorps xmm0, xmm0           ; acumulador = 0.0
3     test esi, esi
4     jz .done
5     mov rcx, rsi               ; rcx = n
6 .loop:
7     movss xmm1, [rdi]          ; carga arr[i]
8     addss xmm0, xmm1           ; sum += arr[i]
9     add rdi, 4                 ; avanzar 4 bytes
10    dec rcx
11    jnz .loop
12 .done:
13    ret
```

3.2. compute_stats (escalar)

Esta función se implementó con **dos pasadas** sobre el arreglo. La primera calcula simultáneamente suma, mínimo y máximo. Con la suma se obtiene la media. La segunda pasada calcula $\sum (x_i - \mu)^2$, de la cual se deriva la varianza poblacional $\sigma^2 = \sum (x_i - \mu)^2 / n$.

La justificación de las dos pasadas es *numérica*: la fórmula “de una pasada” $\sigma^2 = \frac{1}{n} \sum x_i^2 - \mu^2$ sufre de **cancelación catastrófica** cuando $\mu \gg \sigma$ (números grandes con varianza pequeña), porque se restan dos cantidades muy parecidas en float32. La fórmula de dos pasadas es numéricamente estable.

```
1 ; Primera pasada: sum, min, max
2     xorps xmm0, xmm0           ; sum
3     movss xmm1, [rdi]          ; min = arr[0]
4     movss xmm2, [rdi]          ; max = arr[0]
5 .primer_pasada_loop:
6     movss xmm3, [rbx]
7     addss xmm0, xmm3           ; sum += arr[i]
8     comiss xmm3, xmm1
9     jae .skip_min
10    movss xmm1, xmm3           ; min = arr[i]
11 .skip_min:
12    comiss xmm3, xmm2
13    jbe .skip_max
14    movss xmm2, xmm3           ; max = arr[i]
15 .skip_max:
16    ...
17 ; mean = sum / n
18     cvtsi2ss xmm4, r12
19     movss xmm3, xmm0
20     divss xmm3, xmm4
```

```

21     movss [rdx], xmm3
22
23 ; Segunda pasada: sum_sq
24 .segunda_pasada_loop:
25     movss xmm5, [rbx]
26     subss xmm5, xmm3           ; (x - mean)
27     mulss xmm5, xmm5           ; (x - mean)^2
28     addss xmm0, xmm5
29     ...
30 ; var = sum_sq / n
31     divss xmm0, xmm4
32     movss [rcx], xmm0

```

3.3. normalize_array (escalar)

Aplica $\text{out}[i] = (\text{in}[i] - \text{mean})/\text{stddev}$ elemento por elemento. El **caso borde** $\sigma = 0$ (arreglo constante) se maneja explícitamente: en lugar de dividir por cero, se copia la entrada a la salida sin modificar, detectado con `comiss xmm1, xmm2`.

```

1  normalize_array:
2      push rbx
3      push r12
4      test edx, edx
5      jz .done           ; n == 0 -> salir
6      xorps xmm2, xmm2
7      comiss xmm1, xmm2
8      je .zero_stddev    ; sigma == 0 -> copiar
9      mov rbx, rdi
10     mov r12, rsi
11     mov rcx, rdx
12 .norm_loop:
13     movss xmm3, [rbx]
14     subss xmm3, xmm0     ; (in - mean)
15     divss xmm3, xmm1     ; / stddev
16     movss [r12], xmm3
17     add rbx, 4
18     add r12, 4
19     dec rcx
20     jnz .norm_loop
21     jmp .done
22 .zero_stddev:
23     ; copiar in -> out sin modificar
24     ...
25 .done:
26     pop r12
27     pop rbx
28     ret

```

4. Implementación vectorial

4.1. Justificación de AVX2 frente a otras extensiones

Se eligió **AVX2** por tres razones:

1. **Ancho de registro:** 256 bits = 8 float por instrucción, el doble que SSE (4 float).
2. **Disponibilidad:** presente en CPUs Intel desde Haswell (2013) y en AMD desde Excavator (2015), mucho más ubicua que AVX-512 (solo en CPUs de gama alta desde Skylake-X y algunos Zen 4/5).
3. **Frecuencia y complejidad:** AVX-512 introduce registros ZMM de 512 bits, máscaras k0-k7 y suele provocar *downclocking* en algunas microarquitecturas Intel. Ninguna de esas complejidades es necesaria aquí.

4.2. sum_array (vectorial)

El patrón vectorial es: procesar $\lfloor n/8 \rfloor$ bloques de 8 float con `vmovups + vaddps`, reducir horizontalmente los 8 acumuladores parciales a un escalar, y procesar el remanente $n \bmod 8$ con un bucle escalar de cierre.

```
1 sum_array:
2     xor eax, eax                ; i = 0
3     vxorps ymm0, ymm0, ymm0
4     mov ecx, esi
5     and ecx, ~7                ; n redondeado a multiplo de 8
6 .sum_vec_loop:
7     cmp eax, ecx
8     jge .sum_reduce
9     vmovups ymm1, [rdi + rax*4]
10    vaddps ymm0, ymm0, ymm1
11    add eax, 8
12    jmp .sum_vec_loop
13 .sum_reduce:
14     ; reduccion horizontal: 8 carriles -> 1 escalar
15     vextractf128 xmm2, ymm0, 1
16     vaddps xmm0, xmm0, xmm2
17     vhaddps xmm0, xmm0, xmm0
18     vhaddps xmm0, xmm0, xmm0
19 .sum_scalar_tail:
20    cmp eax, esi
21    jge .sum_done
22    vmovss xmm1, [rdi + rax*4]
23    vaddss xmm0, xmm0, xmm1
24    inc eax
25    jmp .sum_scalar_tail
26 .sum_done:
27    vzeroupper
28    ret
```

4.3. Reducción horizontal: estrategia elegida

Para reducir los 8 carriles de un ymm a un escalar se siguió la estrategia **vextractf128 + vaddps + 2×vhaddps**:

1. **vextractf128 xmm2, ymm0, 1**: extrae los 4 carriles altos (4–7) a un xmm.
2. **vaddps xmm0, xmm0, xmm2**: suma carril a carril, dejando 4 sumas parciales en xmm0.
3. **vhaddps xmm0, xmm0, xmm0** (dos veces): cada uno combina pares adyacentes, propagando la suma total al carril 0.

Alternativa considerada: usar solo **vhaddps** sobre ymm sin extraer la mitad alta primero. Se descartó porque en ese caso los 4 carriles bajos terminarían con la suma de la mitad baja y los 4 carriles altos con la suma de la mitad alta, obligando a un **vextractf128** + suma adicional al final de todos modos.

4.4. compute_stats (vectorial)

Estructura análoga a la escalar: dos pasadas vectoriales, con reducciones horizontales al final de cada una y bucle escalar de cierre para el remanente.

```
1 ; --- Primera pasada: sum, min, max ---
2     vxorps ymm0, ymm0, ymm0           ; sum
3     vbroadcastss ymm1, [rdi]          ; min = arr[0] en 8 carriles
4     vbroadcastss ymm2, [rdi]          ; max = arr[0]
5     xor eax, eax
6     mov ecx, esi
7     and ecx, ~7
8 .cs_pass1_loop:
9     cmp eax, ecx
10    jge .cs_pass1_reduce
11    vmovups ymm3, [rdi + rax*4]
12    vaddps ymm0, ymm0, ymm3
13    vminps ymm1, ymm1, ymm3
14    vmaxps ymm2, ymm2, ymm3
15    add eax, 8
16    jmp .cs_pass1_loop
17 .cs_pass1_reduce:
18     ; reduccion horizontal de sum, min y max
19     ...
20 .cs_pass1_tail:
21     ; remanente escalar
22     ...
23 .cs_pass1_done:
24     vcvtsi2ss xmm4, xmm4, esi         ; xmm4 = (float)n
25     vdivss xmm6, xmm0, xmm4           ; mean -> xmm6
26     vmovss [r12], xmm6
27
28 ; --- Segunda pasada: sum_sq = sum((x-mean)^2) ---
29     vxorps ymm0, ymm0, ymm0
30     vbroadcastss ymm7, xmm6           ; mean en 8 carriles (ver Sec. 4.6)
31 .cs_pass2_loop:
32     vmovups ymm5, [rdi + rax*4]
```

```

33     vsubps ymm5, ymm5, ymm7           ; (x - mean)
34     vmulps ymm5, ymm5, ymm5         ; ^2
35     vaddps ymm0, ymm0, ymm5
36     ...
37 .cs_pass2_done:
38     vdivss xmm0, xmm0, xmm4         ; var = sum_sq / n
39     vmovss [r13], xmm0
40     vmovss [r14], xmm1
41     vmovss [r15], xmm2

```

4.5. Manejo del remanente

Si n no es múltiplo de 8, los últimos $n \bmod 8$ elementos se procesan con un bucle escalar de cierre, idéntico al de la versión escalar pero operando sobre las mismas variables acumuladoras (`xmm0`, `xmm1`, `xmm2`) que ya contienen las reducciones horizontales del bucle vectorial. Esto garantiza resultados idénticos a los de la versión puramente escalar (mismo orden de suma para el remanente) y evita accesos fuera de los límites del arreglo.

El número de elementos que entran en el bucle vectorial se calcula con `and ecx, ~7`, que redondea n hacia abajo al múltiplo de 8 más cercano. El remanente es entonces $n - ecx = n \bmod 8$.

4.6. normalize_array (vectorial)

```

1  normalize_array:
2      test edx, edx
3      jz .na_done
4      vxorps xmm2, xmm2, xmm2
5      vucomiss xmm1, xmm2
6      je .na_zero_stddev           ; sigma == 0 -> copiar
7
8      vbroadcastss ymm0, xmm0      ; mean en 8 carriles
9      vbroadcastss ymm1, xmm1      ; stddev en 8 carriles
10     xor eax, eax
11     mov ecx, edx
12     and ecx, ~7
13 .na_norm_loop:
14     cmp eax, ecx
15     jge .na_norm_tail
16     vmovups ymm3, [rbx + rax*4]
17     vsubps ymm3, ymm3, ymm0       ; (in - mean)
18     vdivps ymm3, ymm3, ymm1       ; / stddev
19     vmovups [r12 + rax*4], ymm3
20     add eax, 8
21     jmp .na_norm_loop
22 .na_norm_tail:
23     ; remanente escalar
24     ...

```

4.7. Caso de estudio: bug de corrupción de acumulador

Síntoma. Al correr `norm_vector` contra la implementación de referencia en Python, la suma, la media, el mínimo y el máximo coincidían dentro de la tolerancia (10^{-4}), pero la **varianza** y

la **desviación estándar** fallaban sistemáticamente. El patrón era revelador: para $N = 0$ y para arreglos con varianza real igual a 0 (un solo elemento, o valores constantes) el resultado era correcto “por casualidad”; en cualquier otro caso con varianza distinta de cero, el valor obtenido era incorrecto. Por ejemplo, para un caso con $N = 7$:

	Referencia	Obtenido
var	700,812730	-446,204926
stddev	26,472868	nan

Diagnóstico. El patrón (correcto solo cuando la varianza real es 0) sugería que el bug no estaba en el cálculo de las diferencias al cuadrado, sino en la *división final* por n . Se verificó a mano que el valor obtenido no correspondía a sum_sq/n , sino a $sum_sq/mean$:

$$sum_sq = var_{\text{ref}} \times n = 700,812730 \times 7 \approx 4905,689$$

$$\frac{sum_sq}{mean} = \frac{4905,689}{-10,994250} \approx -446,20$$

que coincide exactamente con el valor obtenido. Esto apuntó a una **corrupción de registro**: el registro que debía contener n como **float** terminó conteniendo la media.

Revisando el código, la causa era que `xmm4/ymm4` se usaba para dos propósitos distintos sin que el segundo preservara el primero:

```

1 .cs_pass1_done:
2     vcvtsi2ss xmm4, xmm4, esi    ; xmm4 = (float)n
3     vdivss    xmm6, xmm0, xmm4    ; mean -> xmm6
4     vmovss    [r12], xmm6
5     vxorps    ymm0, ymm0, ymm0
6     vbroadcastss ymm4, xmm6      ; BUG: pisa ymm4 (que era (float)n)
7     ...
8 .cs_pass2_done:
9     vdivss    xmm0, xmm0, xmm4    ; "var = sum_sq / n", pero xmm4 = mean

```

`vbroadcastss ymm4, xmm6` reescribe los 256 bits completos de `ymm4` (incluyendo `xmm4`) con la media replicada, destruyendo el valor de n .

Confirmación con GDB. Se inspeccionó `xmm4` en dos puntos con breakpoints por línea (las etiquetas locales con punto no son visibles como símbolos de función en GDB):

```

1 (gdb) break stats_vector.asm:<linea de .cs_pass1_done>
2 (gdb) run
3 (gdb) p $xmm4.v4_float
4 $1 = {7, ...} ; xmm4 = (float)n, correcto
5
6 (gdb) break stats_vector.asm:<linea de .cs_pass2_done>
7 (gdb) continue
8 (gdb) p $xmm4.v4_float
9 $2 = {-10.99..., ...} ; BUG: ya no es 7, es la media repetida

```


Corrección. Se usó `ymm7` (no utilizado en ese tramo) para el broadcast:

```

1      ; FIX: ymm7 en vez de ymm4; xmm4 debe preservar (float)n
2      vbroadcastss ymm7, xmm6
3      ...
4      vsubps    ymm5, ymm5, ymm7

```

Tras la corrección, la comparación contra la referencia Python y la comparación cruzada escalar/vectorial coincidieron completamente (9/9 casos), incluyendo los que antes fallaban.

Lección general. Este caso ilustra un riesgo específico de la programación con registros vectoriales de 256 bits: `xmmk` es la mitad baja de `ymm $k$` , por lo que *cualquier* instrucción que escriba `ymm $k$` destruye silenciosamente cualquier valor escalar preservado en `xmm $k$` . La corrección definitiva fue adoptar la disciplina de documentar explícitamente en el encabezado de la función qué registro contiene qué valor y durante qué tramo debe permanecer intacto.

5. Casos de prueba

Se ejecutaron los casos borde obligatorios más otros auxiliares, comparando contra la referencia Python (`verify_reference.py`) y cruzando escalar contra vectorial. La Tabla 2 resume los resultados.

N	Caso	Error rel. máx.	Esc.	Vec.	Cruzada
0	vacío	0	PASA	PASA	PASA
1	un elemento	$2,06 \times 10^{-10}$	PASA	PASA	PASA
7	no múltiplo de 8	$9,34 \times 10^{-8}$	PASA	PASA	PASA
8	múltiplo exacto	$7,08 \times 10^{-7}$	PASA	PASA	PASA
15	no múltiplo de 8	$1,09 \times 10^{-6}$	PASA	PASA	PASA
16	múltiplo exacto	$5,12 \times 10^{-8}$	PASA	PASA	PASA
1000	valores aleatorios	$2,43 \times 10^{-7}$	PASA	PASA	PASA
1000	valores constantes ($\sigma = 0$)	0	PASA	PASA	PASA
100	valores extremos	$1,57 \times 10^{-7}$	PASA	PASA	PASA

Cuadro 2: Casos de prueba verificados. Todos los errores están muy por debajo de la tolerancia 10^{-4} .

Observaciones:

- $N = 0$ y $N = 1$ manejan los casos borde sin división por cero: en $N = 0$ se escriben ceros; en $N = 1$ y en el arreglo constante, $\sigma = 0$ activa la rama de “copiar sin normalizar”.
- Los valores extremos ($\pm 10^6$) siguen funcionando: la varianza alcanza $\sim 3 \times 10^{11}$, dentro del rango de `float32`.
- Escalar y vectorial coinciden entre sí (9/9), lo que confirma que el manejo del remanente y las reducciones horizontales son correctos.

6. Resultados de rendimiento

6.1. Tiempos de ejecución

Se midió el tiempo del kernel con `clock_gettime` en el driver C, promediando 30 repeticiones por tamaño.

N	$t_{escalar}$ (ms) $\pm \sigma$	$t_{vectorial}$ (ms) $\pm \sigma$	Speedup
10^3	$0,0080 \pm 0,0008$	$0,0014 \pm 0,0001$	5,842
10^5	$0,7417 \pm 0,0193$	$0,1095 \pm 0,0068$	6,772
10^6	$7,3074 \pm 0,0808$	$1,3820 \pm 0,0761$	5,288
5×10^7	$252,1542 \pm 5,3312$	$79,3056 \pm 3,8826$	3,180

Cuadro 3: Tiempo promedio del kernel, 30 repeticiones por tamaño.

6.2. Gráfico de speedup vs. N

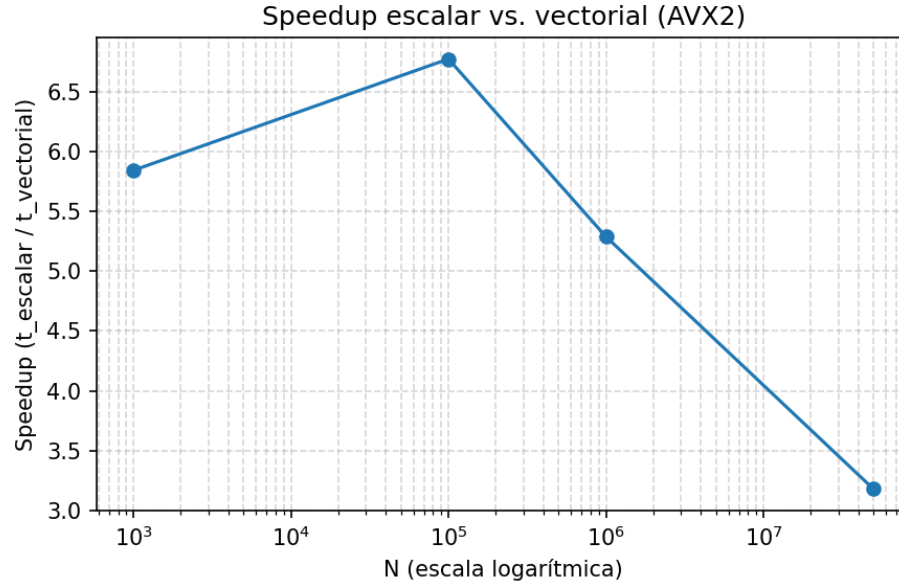


Figura 1: Speedup (escalar/vectorial) vs. tamaño N, escala logarítmica en el eje X.

6.3. perf stat

Se ejecutó `perf stat -e cycles,instructions,cache-misses` sobre ambas versiones con $N = 5 \times 10^7$:

Métrica	Escalar	Vectorial
Ciclos	1 472 905 572	865 568 521
Instrucciones	2 276 627 435	914 737 716
IPC	1.546	1.057
Cache-misses	24 200 085	31 813 733
Tiempo kernel (ms)	250.20	66.86
Tiempo elapsed (s)	0.520	0.334

Cuadro 4: Salida de `perf stat` para $N = 5 \times 10^7$. Los contadores miden el *proceso completo* (incluyendo E/S), mientras que el “tiempo kernel” es el medido con `clock_gettime` alrededor del kernel.

```

ana@ana-Inspiron-3501:~/Arqui_Proyecto_1/proyecto_vectorial$ perf stat -e cycles,instructions,cache-misses ./bin/norm_scalar data/perf_5e7.dat data/out_perf_scalar.dat 1
N = 50000000
Suma = 716428.125000
Media = 0.014329
Varianza = 2914.149902
StdDev = 53.982868
Minimo = -100.000000
Maximo = 100.000000
Tiempo promedio del kernel (1 rep.): 250.199989 ms

Performance counter stats for './bin/norm_scalar data/perf_5e7.dat data/out_perf_scalar.dat 1':

1.472.905.572    cycles
2.276.627.435    instructions
24.200.085       cache-misses

0,520012692 seconds time elapsed

0,271999000 seconds user
0,247999000 seconds sys

ana@ana-Inspiron-3501:~/Arqui_Proyecto_1/proyecto_vectorial$ perf stat -e cycles,instructions,cache-misses ./bin/norm_vector data/perf_5e7.dat data/out_perf_vector.dat 1
N = 50000000
Suma = 716520.812500
Media = 0.014330
Varianza = 3315.316895
StdDev = 57.578789
Minimo = -100.000000
Maximo = 100.000000
Tiempo promedio del kernel (1 rep.): 66.861430 ms

Performance counter stats for './bin/norm_vector data/perf_5e7.dat data/out_perf_vector.dat 1':

865.568.521      cycles
914.737.716      instructions
31.813.733        cache-misses

0,334217903 seconds time elapsed

0,087949000 seconds user
0,245859000 seconds sys

```

Figura 2: Captura de la salida de `perf stat -e cycles,instructions,cache-misses` para $N = 5 \times 10^7$. Arriba: versión escalar (`norm_scalar`) con 250.20 ms de kernel y 1.546 IPC. Abajo: versión vectorial (`norm_vector`) con 66.86 ms de kernel y 1.057 IPC. La diferencia de tiempos del kernel ($\approx 3,74\times$) contrasta con la escasa diferencia del tiempo total del proceso (0,520 s vs. 0,334 s, $\approx 1,56\times$), evidencia del peso de la E/S del archivo de 200 MB.

6.4. Interpretación

El speedup medido con `clock_gettime` decrece conforme crece N : de $5,84\times$ en $N = 10^3$ sube a $6,77\times$ en $N = 10^5$ (donde el cómputo aún domina sobre la memoria), y luego cae a $5,29\times$ en $N = 10^6$ y a $3,18\times$ en $N = 5 \times 10^7$. Este patrón confirma que el kernel es **memory-bound** a partir de cierto tamaño: cuando el arreglo deja de caber en las cachés L1/L2/L3 (a partir de $N \approx 10^6$, 4 MB, que excede la L2 típica y presiona la L3), ambas versiones quedan limitadas por el ancho de banda de memoria principal, y la ventaja de procesar 8 float por instrucción deja de traducirse en speedup proporcional.

Los contadores de `perf stat` para $N = 5 \times 10^7$ refuerzan esta lectura. El IPC de la versión vectorial (1,057) es *menor* que el de la escalar (1,546), pese a que la vectorial ejecuta $2,49\times$ menos instrucciones (914,7M vs. 2276,6M). Esto es la firma característica de un cuello de botella de memoria: al reducir el número de instrucciones, el tiempo que antes se solapaba con trabajo útil de cómputo queda expuesto como ciclos de espera (*stalls*) por datos que aún no llegan de RAM, bajando el IPC efectivo. Consistente con esto, el número de **cache-misses** no disminuyó con la vectorización de hecho aumentó un 31 % (24,2 M vs. 31,8 M) porque el ancho del registro reduce cuántas *instrucciones* se necesitan para pedir los datos, pero no reduce la cantidad total de *bytes* que deben traerse de memoria: se siguen tocando las mismas líneas de caché de 64 bytes para procesar los mismos N floats, en el mismo número de pasadas (suma/min/max, varianza, normalización).

Esto explica directamente por qué el speedup nunca se acerca al $8\times$ teórico de AVX2: según la **Ley de Amdahl**, el beneficio de vectorizar la parte paralelizable del cómputo está acotado por la fracción del tiempo total que depende del ancho de banda de memoria (no paralelizable mediante SIMD), que domina a tamaños grandes. Nótese además que el speedup del *proceso completo* reportado por `perf` ($0,520/0,334 \approx 1,56\times$) es mucho menor que el speedup del *kernel aislado* ($250,20/66,86 \approx 3,74\times$),

porque **perf** mide todo el programa, incluyendo la lectura del archivo de entrada de 200 MB y la escritura de la salida — operaciones de E/S cuyo costo no depende de programa, incluyendo la lectura del archivo de entrada de 200 MB y la escritura de la salida — operaciones de E/S cuyo costo no depende de qué tan rápido sea el kernel vectorizado.

7. Evidencia de la sesión de GDB

Se generó una entrada de prueba con $N = 16$ (`gen_input.py 16 data/test_16.dat random`) y se depuró `norm_vector` con GDB, deteniendo la ejecución justo antes de la instrucción `vaddps` del bucle vectorial de `sum_array`, y nuevamente al final de `normalize_array`.

```

1 (gdb) break stats_vector.asm:34
2 (gdb) run
3 Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:34
4 34          vaddps   ymm0, ymm0, ymm1          ; acumula por carril
5 (gdb) next
6 35          add      eax, 8
7 (gdb) p $ymm0.v8_float
8 $1 = {-33.0938835, 10.0854902, -47.0000877, 49.963829, -88.3387909,
9       6.70975304, 4.44914293, -11.6177187}
10 (gdb) continue
11 Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:34
12 34          vaddps   ymm0, ymm0, ymm1          ; acumula por carril
13 (gdb) next
14 35          add      eax, 8
15 (gdb) p $ymm0.v8_float
16 $2 = {-121.608292, 60.533699, -124.286942, 71.0370865, -146.228882,
17       19.3034439, 49.3984985, 64.7713852}
18 (gdb) delete 1
19 (gdb) break stats_vector.asm:340
20 (gdb) continue
21 Breakpoint 2, normalize_array.na_done () at asm/vector/stats_vector.asm:341
22 341         pop      r13
23 (gdb) p $rsi
24 $3 = 93824992253056
25 (gdb) x/8fw $rsi
26 0x555555559480: -0.496496558  0.355878383  -0.771009505  1.14308977
27 0x555555559490: -1.58704913  0.289240241  0.244615063  -0.072550036
28 (gdb) x/8fw (char*)$rsi+32
29 0x5555555594a0: -1.59051597  1.15265167  -1.36888039  0.57278055
30 0x5555555594b0: -0.985981703  0.405391097  1.04410243  1.66473341

```

```

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Depuración de hilo usando libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:34
34      vaddps ymm0, ymm0, ymm1    ; acumula por carril
(gdb) next
35      add     eax, 8
(gdb) p $ymm0.v8_float
$1 = {-33.0938835, 10.0854902, -47.0000877, 49.963829, -88.3387909, 6.70975304,
4.44914293, -11.6177187}
(gdb) continue
Continuando.

Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:34
34      vaddps ymm0, ymm0, ymm1    ; acumula por carril
(gdb) next
35      add     eax, 8
(gdb) p $ymm0.v8_float
$2 = {-121.608292, 60.533699, -124.286942, 71.0370865, -146.228882, 19.3034439,
49.3984985, 64.7713852}
(gdb) delete 1
(gdb) break stats_vector.asm:340
Punto de interrupción 2 at 0x5555555555087: file asm/vector/stats_vector.asm, line 341.
(gdb) continue
Continuando.

Breakpoint 2, normalize_array.na_done () at asm/vector/stats_vector.asm:341
341      pop     r13
(gdb) p $rsi
$3 = 93824992253056
(gdb) x/8fw $rsi
0x55555555559480: -0.496496558    0.355078383    -0.771009505    1.14368977
0x55555555559490: -1.58704913    0.289240241    0.244615063    -0.072550036
(gdb) x/8fw (char*)$rsi+32
0x555555555594a0: -1.59051597    1.15265167    -1.36088039    0.57278055
0x555555555594b0: -0.985981703    0.405391097    1.04410243    1.66473341

```

Figura 3: Sesión de GDB sobre `norm_vector` con $N = 16$. Arriba: valores de `ymm0` tras dos iteraciones del bucle `vaddps`. Abajo: memoria del arreglo de salida (16 float) tras `normalize_array`.

Lectura de la evidencia: \$1 muestra el acumulador `ymm0` justo después de sumar el primer bloque de 8 float (carril a carril, aún sin combinar entre sí). \$2 muestra el mismo registro tras la segunda y última vuelta del bucle ($N = 16 \Rightarrow$ exactamente 2 iteraciones de 8, sin remanente), con cada carril acumulando la suma de dos elementos ($\text{carril}_k = a_k + a_{k+8}$); sumando los 8 valores de \$2 se obtiene la suma total, que coincide con la impresa por el programa. Las dos lecturas de memoria con `x/8fw` muestran los 16 float de salida ya normalizados: la media de los 16 valores es aproximadamente 0 y están distribuidos en un rango razonable ($\pm 1,6$), consistente con un z-score bien calculado y sin valores `nan` ni corruptos.

Esta sesión ilustra el uso de GDB como herramienta de verificación funcional de bajo nivel: no solo permitió detectar el bug de corrupción de acumulador documentado en la Sección 4.7, sino que sirvió para confirmar visualmente que la reducción horizontal y el almacenamiento final del arreglo normalizado producen los valores esperados en cada carril del registro YMM.

8. Conclusiones, limitaciones y trabajo futuro

Conclusiones. Este proyecto permitió comprobar empíricamente la ventaja de SIMD sobre el modelo escalar en un caso concreto y realista. La versión AVX2 llega a ser entre $5\times$ y $7\times$ más rápida que la escalar en tamaños medianos ($N = 10^3$ a 10^5), y aunque el speedup cae a $\sim 3\times$ en $N = 5 \times 10^7$ por saturación del ancho de banda de memoria, sigue siendo una mejora sustancial. Los cuatro objetivos prácticos del proyecto se cumplieron: (i) escribir kernels SIMD en NASM respetando la convención System V; (ii) manejar correctamente el remanente con un bucle escalar de cierre; (iii) depurar errores típicos de bajo nivel con GDB; y (iv) interpretar cuantitativamente el rendimiento a la luz de la Ley de Amdahl y del ancho de banda de memoria.

Limitaciones observadas. La principal limitación fue la brecha entre el speedup teórico de $8\times$ y el medido, explicada por dos factores: (i) la parte no paralelizable mediante SIMD del tiempo total (accesos a memoria, E/S del archivo de entrada), que acota el beneficio máximo; y (ii) la

dependencia serial de `vaddps` dentro del bucle vectorial, que crea una cadena de dependencias entre iteraciones y limita el paralelismo a nivel de instrucción. Adicionalmente, el bug de corrupción de acumulador (`xmm4/ymm4` reutilizado) evidenció lo fácil que es introducir errores silenciosos al mezclar registros escalares y vectoriales en la misma función.

A. Diagrama de arquitectura

El diagrama de arquitectura y flujo de control se entrega como archivo independiente (diagrama_architectura.drawio.pdf), según lo requerido por la Sección 3.a del enunciado. Incluye las cuatro piezas exigidas:

1. **Diagrama de bloques** de la arquitectura de software (driver en C, `libstats_scalar.asm`, `libstats_vector.asm`), con la convención de llamada usada entre C y NASM (System V AMD64 ABI: argumentos en `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` y `xmm0/xmm1`; valor de retorno en `xmm0`).
2. **Diagrama de flujo de control** de cada kernel, mostrando la inicialización del acumulador, la condición del bucle vectorizado (iteraciones de 8 en 8), la transición al bucle remanente (escalar) y la condición de salida.
3. **Tabla de asignación de registros** para el kernel crítico (`compute_stats`), indicando qué contiene cada uno (`ymm0`, `ymm1`, `ymm2`, `ymm3`, `ymm4`, `ymm5`, `ymm6`, `ymm7`, `rdi`, `rsi`, `r12-r15`) en cada etapa del algoritmo.
4. **Diagrama de memoria** mostrando la disposición de los arreglos de entrada y salida, la alineación de la dirección base a un múltiplo de 32 bytes, y cómo avanza el puntero en cada iteración (+4 bytes en escalar, +32 bytes en vectorial).